CS 231
Computer Architecture

Dr. Alisher ikramov

Spring, 2025-2026

Introduction

- Course will have Lectures, Labs (on computers with Windows), Exams.
- **Graded assignments** are Exams and in-Lab work
- **Attendance policy**: if a student misses at least 10 minutes of the lesson, they are marked as absent for the whole lesson (**Lesson = 2 hours**)
- Labs will cover creating computer programs using Assembler (NASM), parallel programming.
- Lectures will cover theoretical aspects of Computer Architecture.

Introduction

Assignments' weights:

- in-Lab work 20 %,
- Mid-term Exam 40 %,
- Oral Final exam 40 %

Oral Final Exam

- The Oral Final exam will take place in **Week 15** during **Lectures** and **Tutorials**. Students who scored above the threshold in Mid-term will be examined during the Lectures, others will be examined during their assigned Tutorials. Students will be given the list of the topics.
- Students randomly select a sheet with **2 big** questions, prepare for **30** minutes (with the written response), answer them to the panel, and receive **3 big extra** questions and **unlimited** number of **small extra** questions. Each **big** question is worth of **20** points, small questions can adjust those points of the related big question.

Course Perspective

- **Our Course is Programmer-Centric**
  - Purpose is to show that by knowing more about the underlying system, one can be more effective as a programmer
  - Enable you to
    - Write programs that are more reliable and efficient
    - Incorporate features that require hooks into OS
      - E.g., concurrency, signal handlers
  - Cover material in this course that you won't see elsewhere

Textbooks

1. Hennessy, J. L., and D. A. Patterson. Computer Architecture: A Quantitative Approach, 7th ed., 2025, ISBN: 9780443154072
2. David A Patterson, John L Hennessy. Computer Organization and Design / The Hardware Software Interface, 5th Edition, 2014, ISBN 0124077269
3. Randal E. Bryant and David R. O'Hallaron, "Computer Systems: A Programmer's Perspective, Second Edition" (CS:APP2e), Prentice Hall, 2011

Cheating

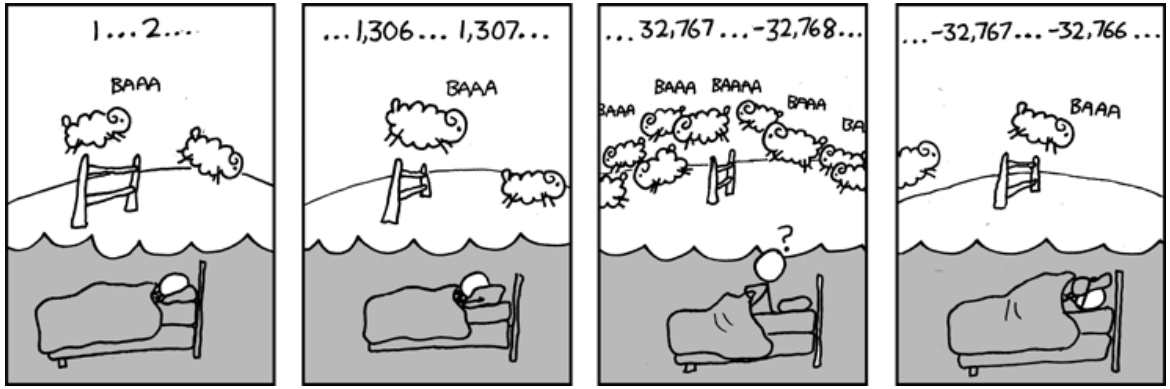
- **What is cheating?**
  - Sharing code: by copying, retyping, **looking at**, or supplying a file
  - Coaching: helping your friend to write a lab, line by line
  - Altering attendance records in any way
  - Copying code from previous course or from elsewhere on WWW
    - Only allowed to use code we supply, or from CS:APP website
- **What is NOT cheating?**
  - Explaining how to use systems or tools
  - Helping others with high-level design issues
- **Penalty for cheating:**
  - Removal from course with failing grade
  - Permanent mark on your record

Course Theme:
Abstraction Is Good But Don't Forget Reality

- **Most CS and CE courses emphasize abstraction**
  - Abstract data types
  - Asymptotic analysis
- **These abstractions have limits**
  - Especially in the presence of bugs
  - Need to understand details of underlying implementations
- **Useful outcomes**
  - Become more effective programmers
    - Able to find and eliminate bugs efficiently
    - Able to understand and tune for program performance
  - Prepare for later "systems" classes in CS & ECE
    - Compilers, Operating Systems, Networks, Embedded Systems, Storage Systems, etc.

Great Reality #1:
Ints are not Integers, Floats are not Reals

- **Example 1: Is $x^2 \geq 0$?**
  - Float's: Yes!



  - Int's:
    - 40000 * 40000 ☞ 1600000000
    - 50000 * 50000 ☞ ??
- **Example 2: Is (x + y) + z = x + (y + z)?**
  - Unsigned & Signed Int's: Yes!
  - Float's:
    - (1e20 + -1e20) + 3.14 --> 3.14
    - 1e20 + (-1e20 + 3.14) --> ??

Source: xkcd.com/571

Computer Arithmetic

- **Does not generate random values**
  - Arithmetic operations have important mathematical properties
- **Cannot assume all "usual" mathematical properties**
  - Due to finiteness of representations
  - Integer operations satisfy "ring" properties
    - Commutativity, associativity, distributivity
  - Floating point operations satisfy "ordering" properties
    - Monotonicity, values of signs
- **Observation**
  - Need to understand which abstractions apply in which contexts
  - Important issues for compiler writers and serious application programmers

Great Reality #2: You've Got to Know Assembly

- **Chances are, you'll never write programs in assembly**
  - Compilers are much better & more patient than you are
- **But: Understanding assembly is key to machine-level execution model**
  - Behavior of programs in presence of bugs
    - High-level language models break down
  - Tuning program performance
    - Understand optimizations done / not done by the compiler
    - Understanding sources of program inefficiency
  - Implementing system software
    - Compiler has machine code as target
    - Operating systems must manage process state
  - Creating / fighting malware
    - x86 assembly is the language of choice!

Great Reality #3: Memory Matters

Random Access Memory Is an Unphysical Abstraction

- **Memory is not unbounded**
  - It must be allocated and managed
  - Many applications are memory dominated
- **Memory referencing bugs especially pernicious**
  - Effects are distant in both time and space
- **Memory performance is not uniform**
  - Cache and virtual memory effects can greatly affect program performance
  - Adapting program to characteristics of memory system can lead to major speed improvements

Memory Referencing Bug Example

```
double fun(int i)
{
  volatile double d[1] = {3.14};
  volatile long int a[2];
  a[i] = 1073741824; /* Possibly out of bounds */
  return d[0];
}
```

```
fun(0)  ☞   3.14
fun(1)  ☞   3.14
fun(2)  ☞   3.1399998664856
fun(3)  ☞   2.0000061035156
fun(4)  ☞   3.14, then segmentation fault
```

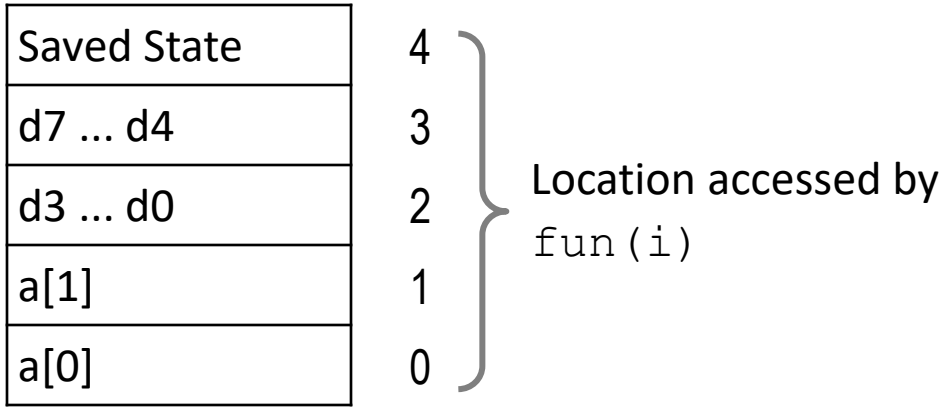
- **Result is architecture specific**

Memory Referencing Bug Example

```
double fun(int i)
{
  volatile double d[1] = {3.14};
  volatile long int a[2];
  a[i] = 1073741824; /* Possibly out of bounds */
  return d[0];
}
```

```
fun(0)  ☙   3.14
fun(1)  ☙   3.14
fun(2)  ☙   3.1399998664856
fun(3)  ☙   2.0000061035156
fun(4)  ☙   3.14, then segmentation fault
```

Explanation:



Memory Referencing Errors

- **C and C++ do not provide any memory protection**
  - Out of bounds array references
  - Invalid pointer values
  - Abuses of malloc/free
- **Can lead to nasty bugs**
  - Whether or not bug has any effect depends on system and compiler
  - Action at a distance
    - Corrupted object logically unrelated to one being accessed
    - Effect of bug may be first observed long after it is generated
- **How can I deal with this?**
  - Program in Java, Ruby or ML
  - Understand what possible interactions may occur
  - Use or develop tools to detect referencing errors (e.g. Valgrind)

Great Reality #4: There's more to performance than asymptotic complexity

- **Constant factors matter too!**
- **And even exact op count does not predict performance**
  - Easily see 10:1 performance range depending on how code written
  - Must optimize at multiple levels: algorithm, data representations, procedures, and loops
- **Must understand system to optimize performance**
  - How programs compiled and executed
  - How to measure program performance and identify bottlenecks
  - How to improve performance without destroying code modularity and generality

Memory System Performance Example

```
void copyij(int src[2048][2048],
            int dst[2048][2048])
{
  int i,j;
  for (i = 0; i < 2048; i++)
    for (j = 0; j < 2048; j++)
      dst[i][j] = src[i][j];
}
```

```
void copyji(int src[2048][2048],
            int dst[2048][2048])
{
  int i,j;
  for (j = 0; j < 2048; j++)
    for (i = 0; i < 2048; i++)
      dst[i][j] = src[i][j];
}
```

21 times slower (Pentium 4)

- **Hierarchical memory organization**
- **Performance depends on access patterns**
  - Including how step through multi-dimensional array

Great Reality #5:
Computers do more than execute programs

- **They need to get data in and out**
  - I/O system critical to program reliability and performance

- **They communicate with each other over networks**
  - Many system-level issues arise in presence of network
    - Concurrent operations by autonomous processes
    - Coping with unreliable media
    - Cross platform compatibility
    - Complex performance issues

Code Security Example

```
/* Kernel memory region holding user-accessible data */
#define KSIZE 1024
char kbuf[KSIZE];

/* Copy at most maxlen bytes from kernel region to user buffer */
int copy_from_kernel(void *user_dest, int maxlen) {
    /* Byte count len is minimum of buffer size and maxlen */
    int len = KSIZE < maxlen ? KSIZE : maxlen;
    memcpy(user_dest, kbuf, len);
    return len;
}
```

- Similar to code found in FreeBSD's implementation of getpeername
- There are legions of smart people trying to find vulnerabilities in programs

Typical Usage

```
/* Kernel memory region holding user-accessible data */
#define KSIZE 1024
char kbuf[KSIZE];

/* Copy at most maxlen bytes from kernel region to user buffer */
int copy_from_kernel(void *user_dest, int maxlen) {
    /* Byte count len is minimum of buffer size and maxlen */
    int len = KSIZE < maxlen ? KSIZE : maxlen;
    memcpy(user_dest, kbuf, len);
    return len;
}
```

```
#define MSIZE 528

void getstuff() {
    char mybuf[MSIZE];
    copy_from_kernel(mybuf, MSIZE);
    printf("%s\n", mybuf);
}
```

Malicious Usage

```
/* Kernel memory region holding user-accessible data */
#define KSIZE 1024
char kbuf[KSIZE];

/* Copy at most maxlen bytes from kernel region to user buffer */
int copy_from_kernel(void *user_dest, int maxlen) {
    /* Byte count len is minimum of buffer size and maxlen */
    int len = KSIZE < maxlen ? KSIZE : maxlen;
    memcpy(user_dest, kbuf, len);
    return len;
}
```

```
#define MSIZE 528

void getstuff() {
    char mybuf[MSIZE];
    copy_from_kernel(mybuf, -MSIZE);
    . . .
}
```

Assembly Code Example

- **Time Stamp Counter**
  - Special 64-bit register in Intel-compatible machines
  - Incremented every clock cycle
  - Read with rdtsc instruction
- **Application**
  - Measure time (in clock cycles) required by procedure

```
double t;
start_counter();
P();
t = get_counter();
printf("P required %f clock cycles\n", t);
```

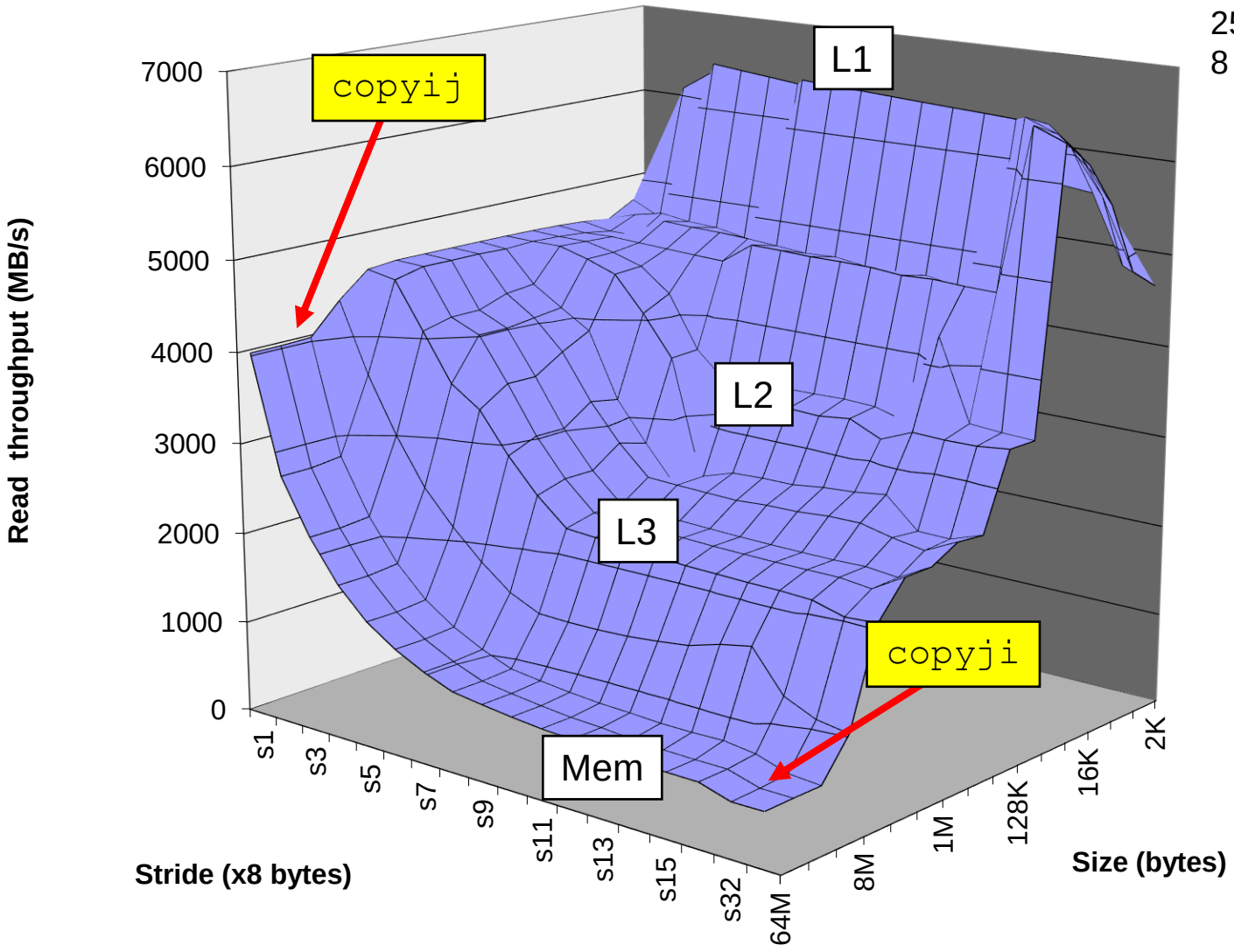
Code to Read Counter

- Write small amount of assembly code using GCC's asm facility
- Inserts assembly code into machine code generated by compiler

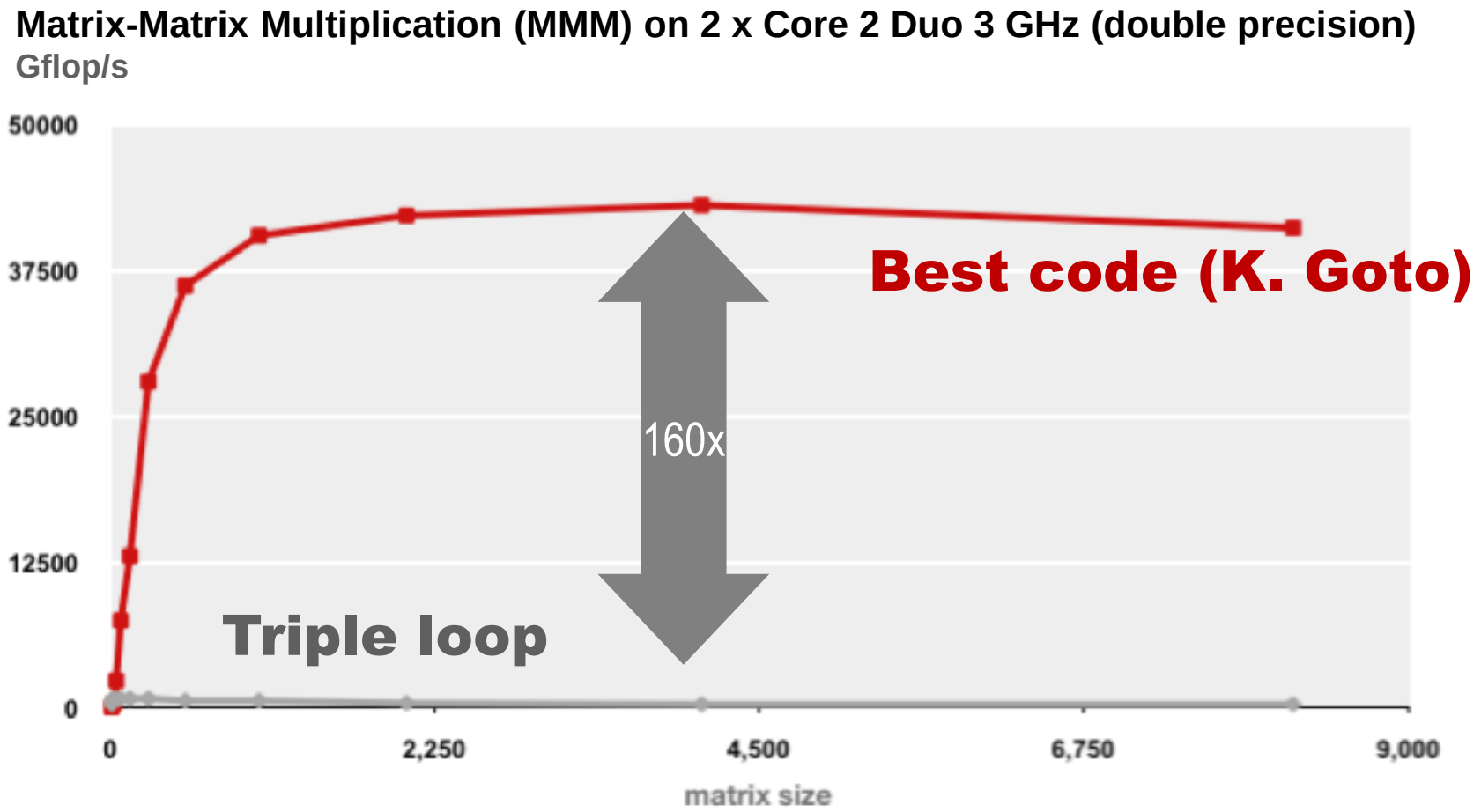
```
static unsigned cyc_hi = 0;
static unsigned cyc_lo = 0;

/* Set *hi and *lo to the high and low order bits
   of the cycle counter.
*/
void access_counter(unsigned *hi, unsigned *lo)
{
    asm("rdtsc; movl %%edx,%0; movl %%eax,%1"
        : "=r" (*hi), "=r" (*lo)
        :
        : "%edx", "%eax");
}
```

The Memory Mountain

Intel Core i7
2.67 GHz
32 KB L1 d-cache
256 KB L2 cache
8 MB L3 cache

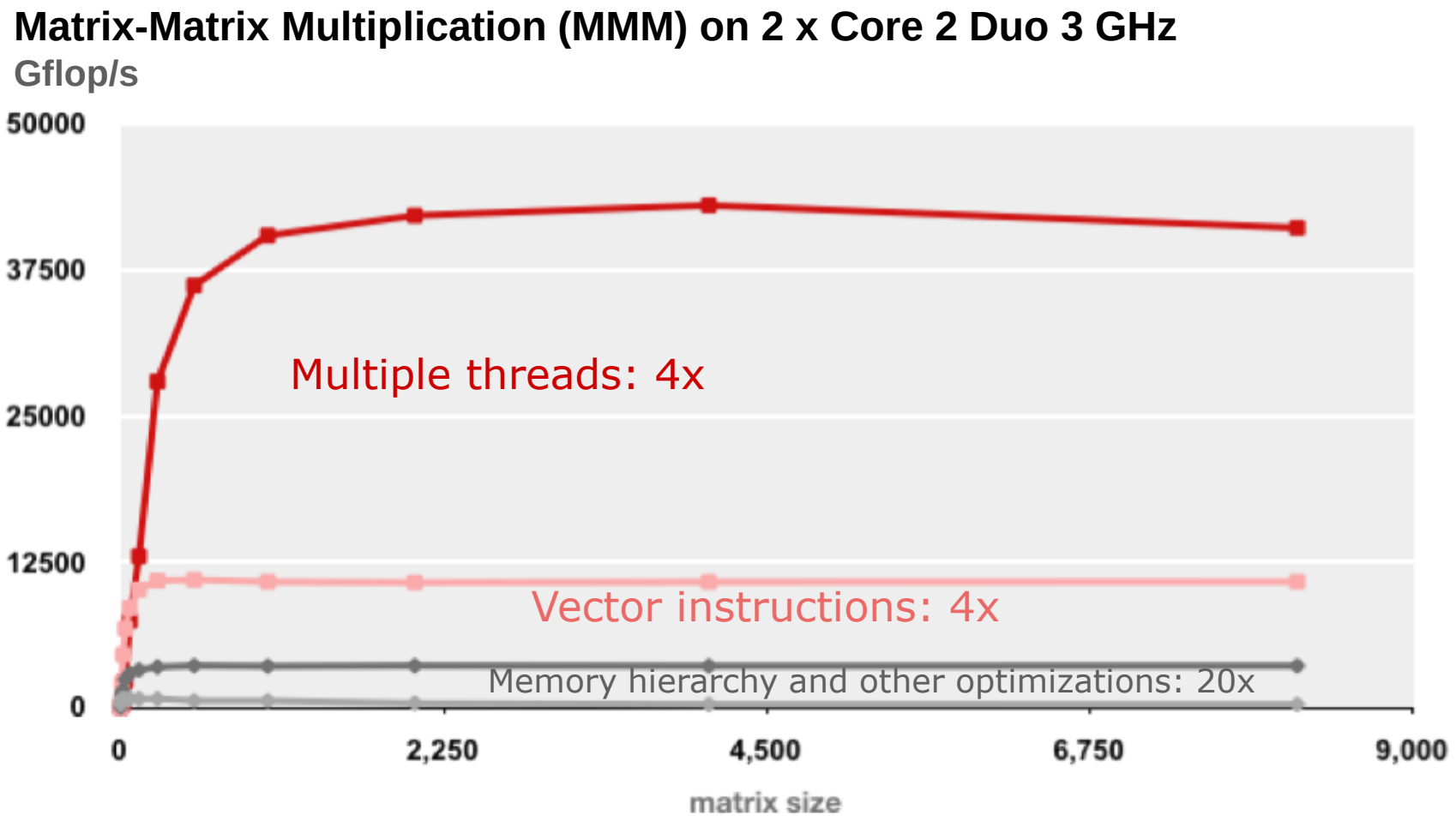


Example Matrix Multiplication



- Standard desktop computer, vendor compiler, using optimization flags
- Both implementations have exactly the same operations count ($2n^3$)
- What is going on?

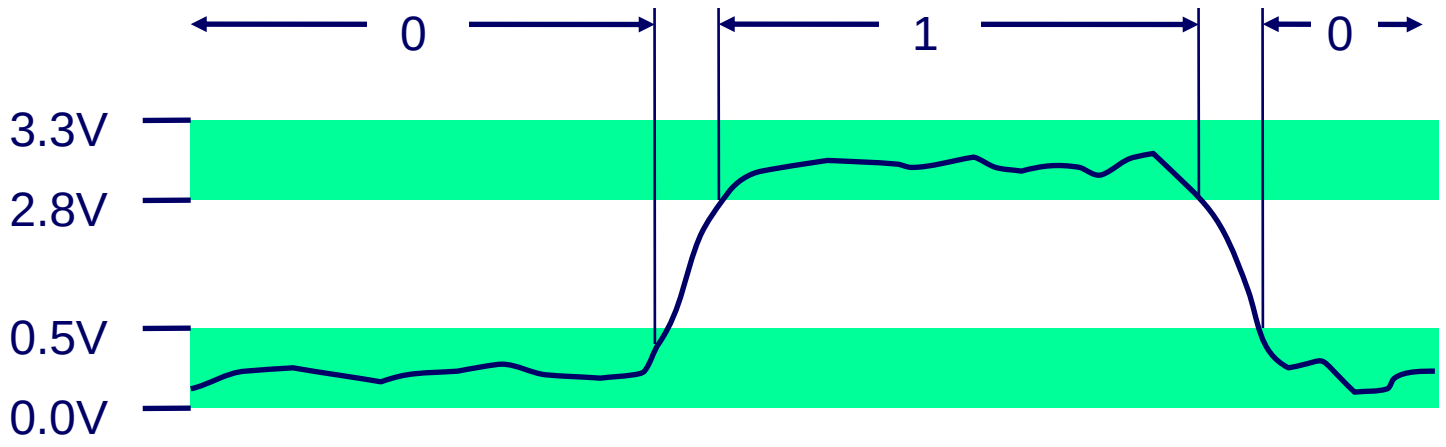
MMM Plot: Analysis



- Reason for 20x: Blocking or tiling, loop unrolling, array scalarization, instruction scheduling, search to find best choice
- ***Effect: fewer register spills, L1/L2 cache misses, and TLB misses***

Binary Representations

- **Base 2 Number Representation**
  - Represent 15213_{10} as 11101101101101_2
  - Represent 1.20_{10} as $1.0011001100110011[0011]\ldots_2$
  - Represent 1.5213×10^4 as $1.1101101101101_2 \times 2^{13}$
- **Electronic Implementation**
  - Easy to store with bistable elements
  - Reliably transmitted on noisy and inaccurate wires



Encoding Byte Values

- **Byte = 8 bits**
  - Binary 00000000_2 to 11111111_2
  - Decimal: 0_{10} to 255_{10}
  - Hexadecimal 00_{16} to FF_{16}
    - Base 16 number representation
    - Use characters ‘0’ to ‘9’ and ‘A’ to ‘F’
    - Write $FA1D37B_{16}$ in C as
      - 0xFA1D37B
      - 0xfa1d37b

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Data Representations

C Data Type	Typical 32-bit	Intel IA32	x86-64
char	1	1	1
short	2	2	2
int	4	4	4
long	4	4	8
long long	8	8	8
float	4	4	4
double	8	8	8
long double	8	10/12	10/16
pointer	4	4	8

General Boolean Algebras

- **Operate on Bit Vectors**
  - Operations applied bitwise

```
  01101001     01101001     01101001
& 01010101   | 01010101   ^ 01010101   ~ 01010101
  --------     --------     --------     --------
  01000001     01111101     00111100     10101010
```

- **All of the Properties of Boolean Algebra Apply**

Example: Representing & Manipulating Sets

- **Representation**
  - Width w bit vector represents subsets of {0, …, w–1}
  - $a_j = 1$ if $j \in A$
    - 01101001 { 0, 3, 5, 6 }
    - *76543210*
    - 01010101 { 0, 2, 4, 6 }
    - *76543210*
- **Operations**

&	Intersection	01000001	{ 0, 6 }
|	Union	01111101	{ 0, 2, 3, 4, 5, 6 }
^	Symmetric difference	00111100	{ 2, 3, 4, 5 }
~	Complement	10101010	{ 1, 3, 5, 7 }

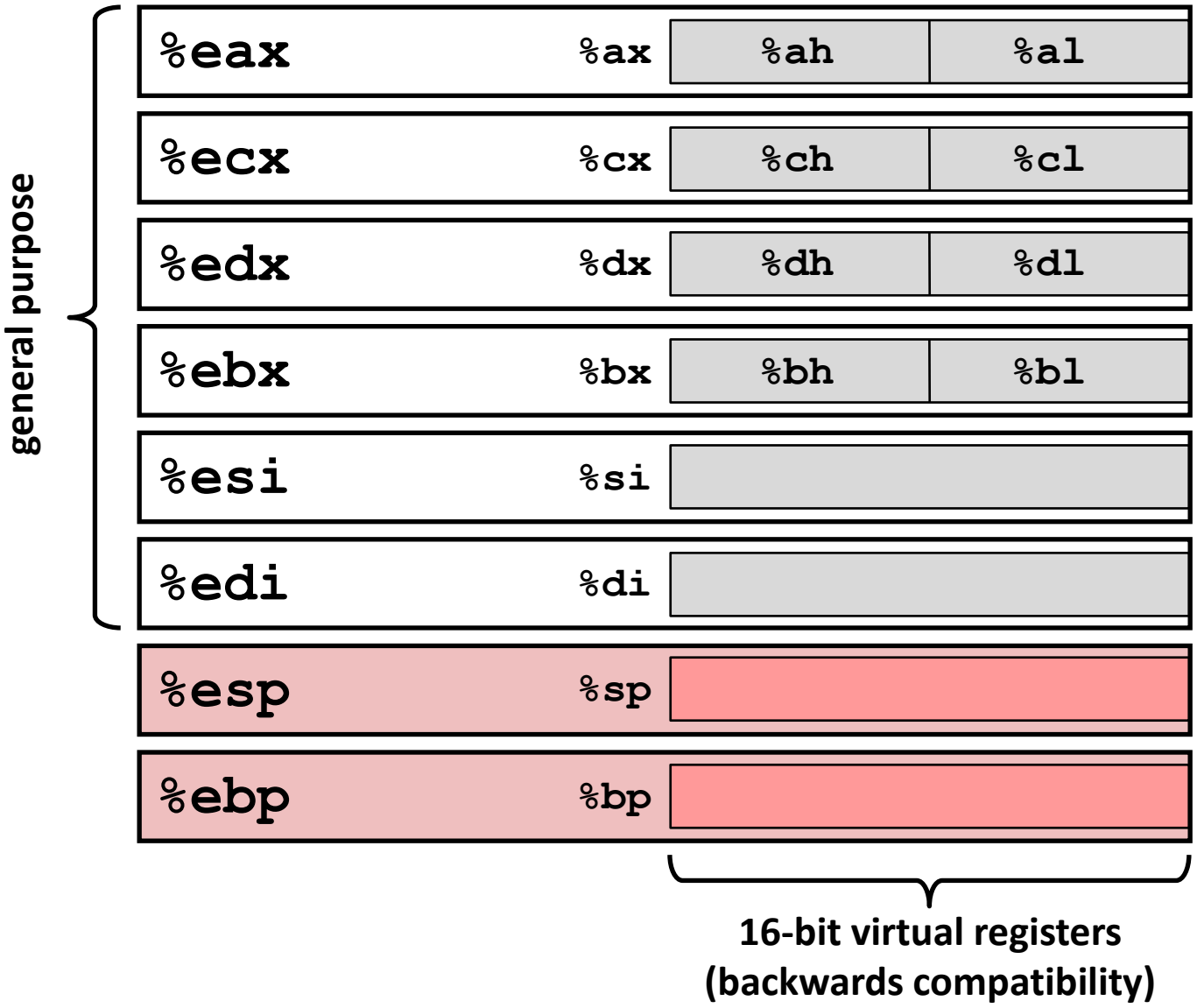
Why Assembly?

- **The Bridge:** Assembly is the thinnest abstraction over the hardware.
- **Why NASM?**
  - Uses Intel Syntax (destination first: mov dst, src).
  - Widely used in Linux and Windows development.
  - Supports a vast range of output formats (ELF64, Mach-O, Win64).
- **NASM Philosophy:** It is "Netwide." It aims to be portable and efficient, without the heavy "auto-intelligence" of MASM/TASM.

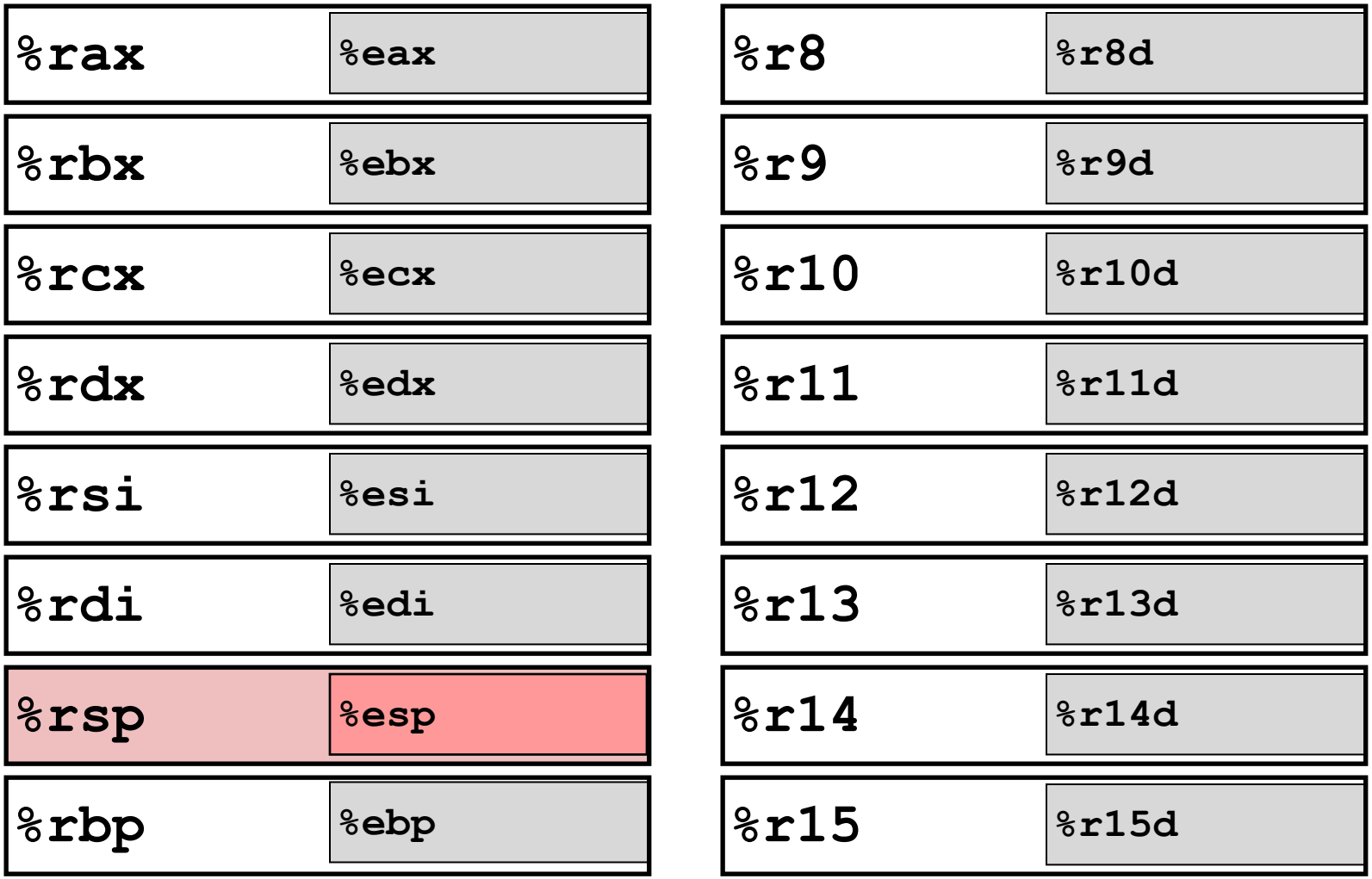
Anatomy of a NASM Source File

- **Sections: NASM organizes code into logical segments:**
  - section **.data**: For initialized global variables (e.g., constants, strings).
  - section **.bss**: For uninitialized data (Block Started by Symbol).
  - section **.text**: Where the actual machine instructions reside.
- **Labels: Names followed by a colon (e.g., _start:) representing memory addresses.**
- **Global Directive: global _start — tells the linker the entry point.**

Integer Registers (IA32)



x86-64 Integer Registers



- Extend existing registers. Add 8 new ones.
- Make `%ebp`/`%rbp` general purpose

NASM Syntax Basics

- **Format:** `label: instruction operands ; comment`
- **Constants:**
  - Decimal: 200
  - Hex: 0xc8 or 0ach
  - Binary: 11001000b
- **Expressions: NASM allows basic math in labels (e.g., mov rax, label + 10).**
- **Case Sensitivity: Instructions are case-insensitive, but labels are case-sensitive by default.**

Data Movement: MOV and LEA

- **MOV instruction: The workhorse.**
  - mov rax, 42 (Immediate to register)
  - mov [var], rax (Register to memory)
- **The Bracket Rule: In NASM, [address] means "the contents of memory at this address.“**
  - mov rax, my_var → rax gets the address of the variable.
  - mov rax, [my_var] → rax gets the value stored there.
- **LEA (Load Effective Address): Calculates an address without accessing memory.**
  - lea rdi, [rbx + rcx * 4] (Commonly used for fast pointer math).

Defining Data (Directives)

- **Initialized (.data):**
  - db (byte), dw (word), dd (dword), dq (qword).
  - Example: msg db 'Hello', 10, 0
- **Uninitialized (.bss):**
  - resb, resw, resd, resq (reserve N units).
  - Example: buffer resb 64
- **Equates:**
  - STDOUT equ 1 — Constant definition (pre-processor).

Arithmetic and Logic

- **Basic Ops: add, sub, imul, idiv.**
- **Logic: and, or, xor, not.**
- **XOR Trick: xor rax, rax is the standard way to set a register to zero (way shorter and faster than mov rax, 0).**
- **The Flags Register (RFLAGS):**
  - ZF (Zero Flag), SF (Sign Flag), CF (Carry Flag).
  - Updated automatically by arithmetic instructions.

Control Flow

- **Unconditional: jmp label**
- **Conditional: Based on CMP (Compare) or TEST.**
  - cmp rax, rbx
  - je label (Jump if equal / ZF=1)
  - jne label (Jump if not equal)
  - jg / jl (Signed Jump Greater/Less)
  - ja / jb (Unsigned Jump Above/Below)
- **Loops: While NASM has a loop instruction, manual dec and jnz are usually preferred for performance.**

Effective Addressing (Memory Access)

- **The Formula: [base + index * scale + displacement]**
- **Constraints:**
  - Base/Index: Any GPR.
  - Scale: Must be 1, 2, 4, or 8.
  - Displacement: Constant offset.
- **Example: mov eax, [rbx + rcx*4 + 8]**
  - Useful for array access (Base = array start, index = element index, scale = element size).

The Stack and Procedures

- **Stack: LIFO structure in memory managed by rsp.**
  - push rax: Decrement rsp, store value.
  - pop rbx: Load value, increment rsp.
- **Procedures:**
  - call function_name: Pushes the Return Address (RIP) to the stack.
  - ret: Pops the address from the stack and jumps to it.
- **Calling Convention (Linux x86-64):**
  - Arguments passed in: rdi, rsi, rdx, rcx, r8, r9

Toolchain: From ASM to EXE

- **Assembly: Use NASM to create an object file.**
  - nasm -f win64 program.asm -o program.o
- **Linking: Use ld (GoLink or gcc) to create the executable.**
  - ld/GoLink/gcc program.o -o program.exe
- **Execution: program.exe**
- **Debugging:** Use objdump (from mingw) to inspect the machine code.

Bit-Level Operations in C

- **Operations &, |, ~, ^ Available in C**
  - Apply to any "integral" data type
    - long, int, short, char, unsigned
  - View arguments as bit vectors
  - Arguments applied bit-wise
- **Examples (Char data type)**
  - ~0x41 ↝ 0xBE
    - $\sim 01000001_2 \rightsquigarrow 10111110_2$
  - ~0x00 ↝ 0xFF
    - $\sim 00000000_2 \rightsquigarrow 11111111_2$
  - 0x69 & 0x55 ↝ 0x41
    - $01101001_2 \;\&\; 01010101_2 \rightsquigarrow 01000001_2$
  - 0x69 | 0x55 ↝ 0x7D
    - $01101001_2 \mid 01010101_2 \rightsquigarrow 01111101_2$

Contrast: Logic Operations in C

- **Contrast to Logical Operators**
  - &&, ||, !
    - View 0 as “False”
    - Anything nonzero as “True”
    - Always return 0 or 1
    - Early termination
- **Examples (char data type)**
  - !0x41 ↝0x00
  - !0x00 ↝0x01
  - !!0x41 ↝0x01

  - 0x69 && 0x55 ↝0x01
  - 0x69 || 0x55 ↝0x01
  - p && *p (avoids null pointer access)

Contrast: Logic Operations in C

- **Contrast to Logical Operators**
  - &&, ||, !
    - View 0 as "Fal
    - Anything nonze
    - Alway
    - Early
- **Example**
  - !0x41 ↝
  - !0x00 ↝
  - !!0x41 ↝
  - 0x69 && 0x55 ↝0x01
  - 0x69 || 0x55 ↝0x01
  - p && *p (avoids null pointer access)

Watch out for && vs. & (and || vs. |)… one of the more common oopsies in C programming

Shift Operations

- **Left Shift: x << y**
  - Shift bit-vector x left y positions
    - Throw away extra bits on left
    - Fill with 0's on right
- **Right Shift: x >> y**
  - Shift bit-vector x right y positions
    - Throw away extra bits on right
  - Logical shift
    - Fill with 0's on left
  - Arithmetic shift
    - Replicate most significant bit on left
- **Undefined Behavior**
  - Shift amount < 0 or ≥ word size

Argument x	01100010
<< 3	00010000
Log. >> 2	00011000
Arith. >> 2	00011000

Argument x	10100010
<< 3	00010000
Log. >> 2	00101000
Arith. >> 2	11101000